

# $\Delta \Sigma$ 変調器のADC

ISHI会

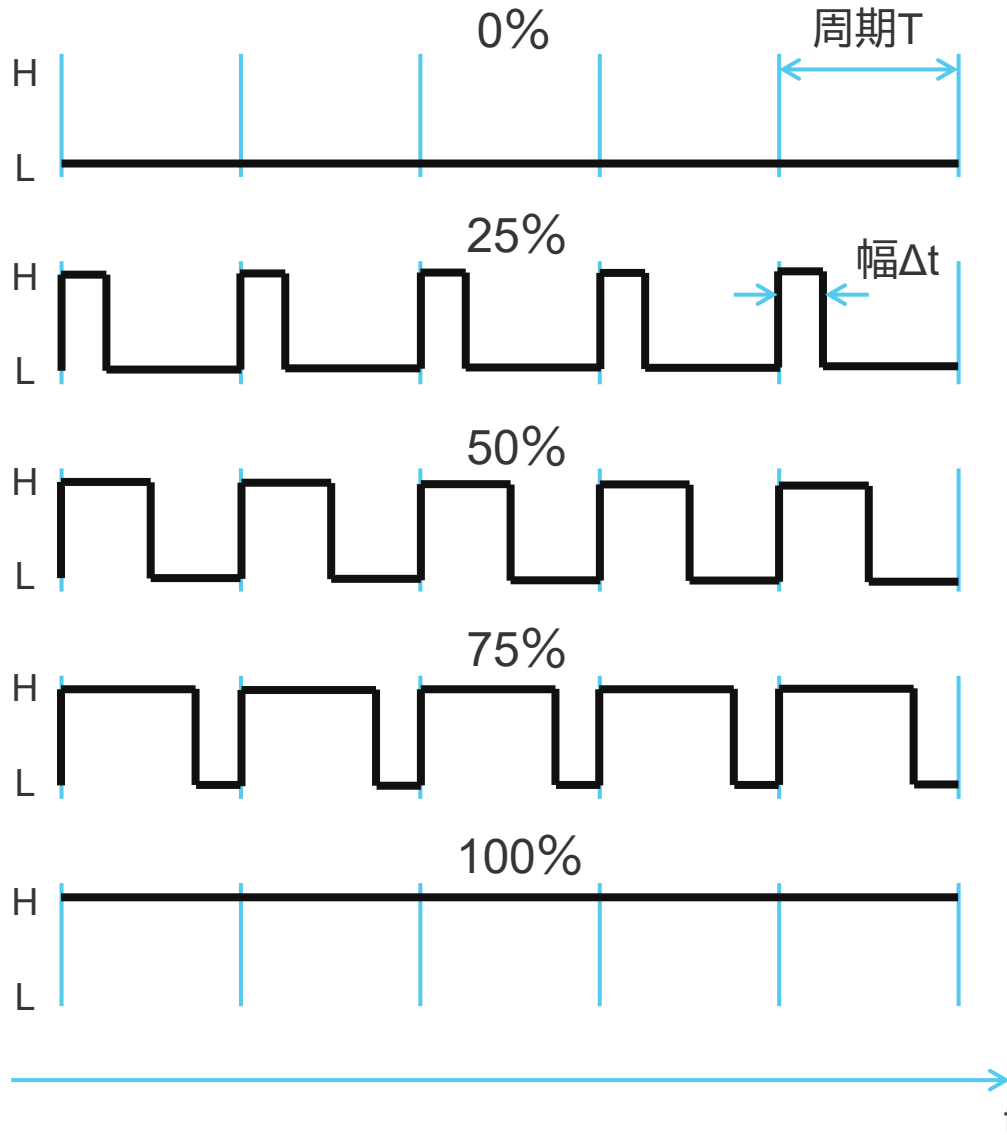
フルデジタルで音声を操るためのロジック回路 4回目

# 目次

- 先週の回答
  - PWMを使ってLEDの調光をします。
- $\Delta$ 変調と $\Delta\Sigma$ 変調
- $\Delta\Sigma$ 変調のシミュレーション
- $\Delta\Sigma$ 変調のFPGAへの実装

# PWMを用いたLEDの調光

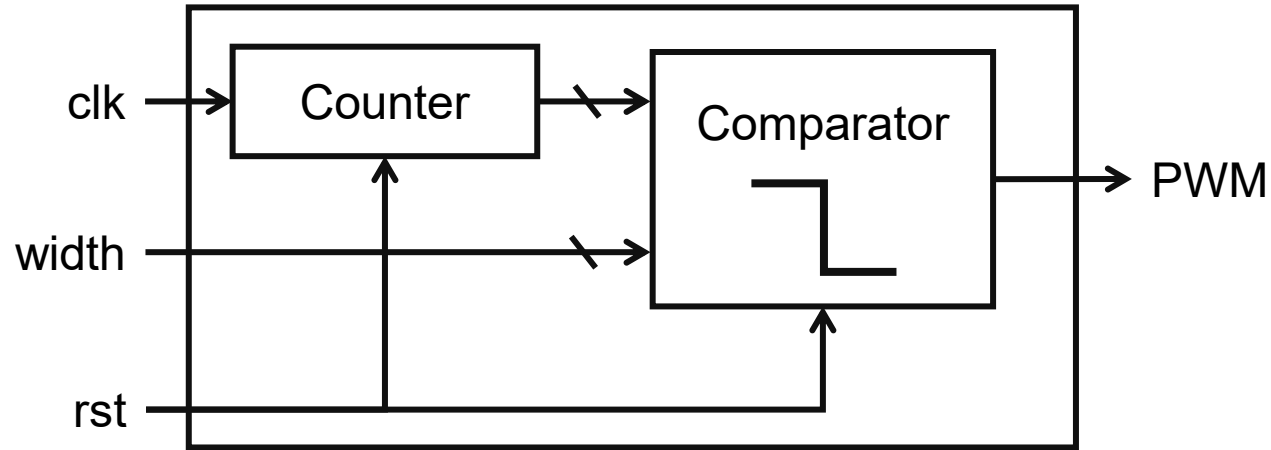
# PWM (Pulse Width Modulation)とは？



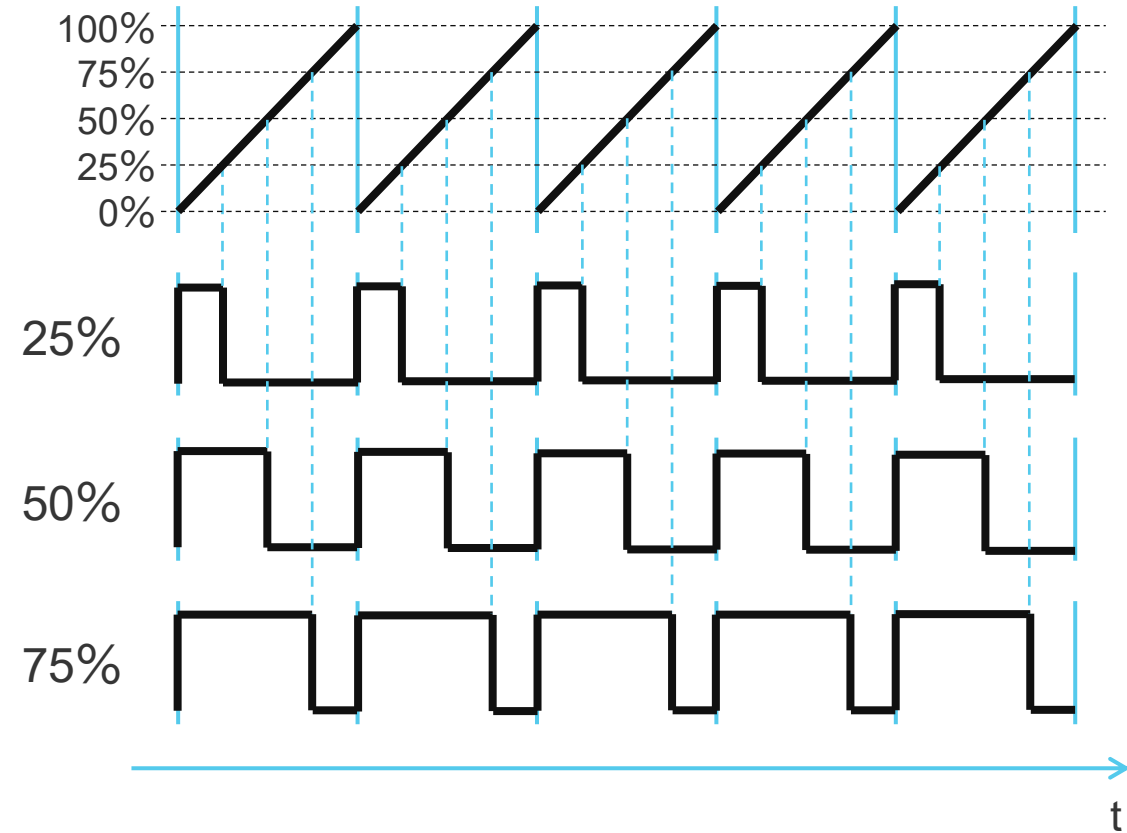
- パルスの幅 $\Delta t$ を変調することで情報を表現する方法
- 周期 $T$ は一定
- 波形を平均することでアナログ電圧が得られる
- **PWM波形でLEDを点灯することで、LEDを調光できる**

# PWMの作り方

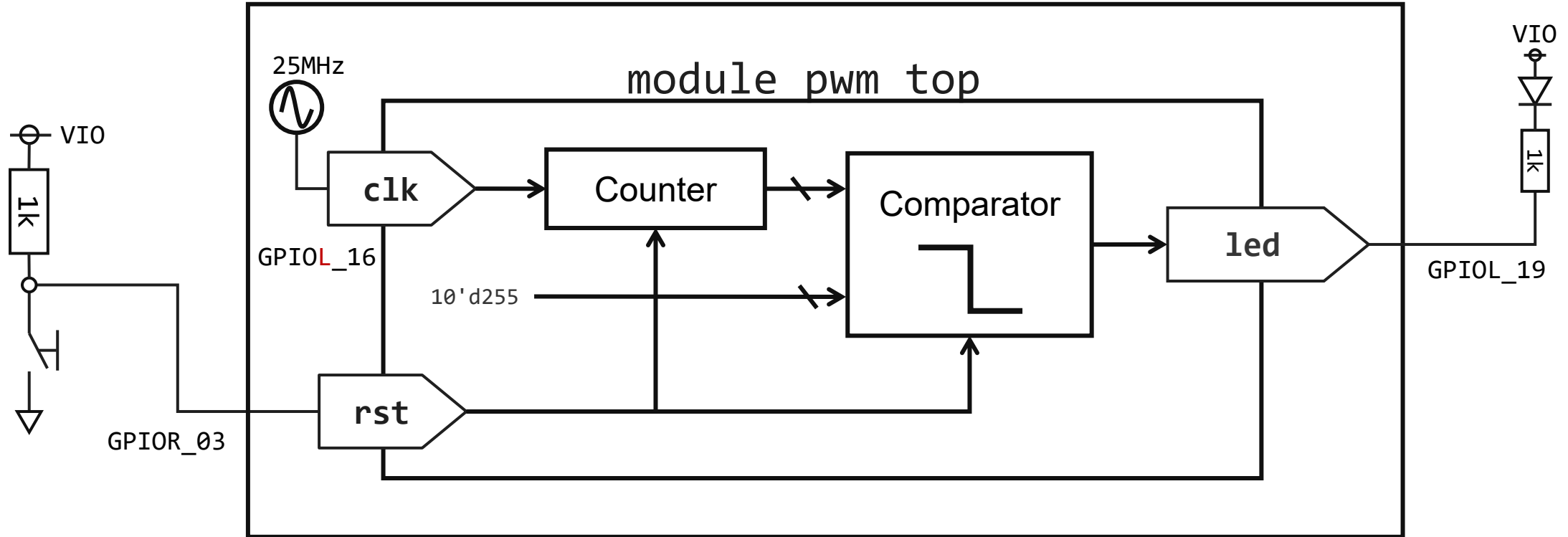
## PWMモジュール



- PWMを作るにはカウンタと比較器を組み合わせる。
- カウンタはカウントアップしていて、最大値に到達したら初期値に戻る。
- カウンタの値と目標値を比較して、超えていたら出力をLに、超えていなければ出力をHにする。



# 回路



# Verilogコード

```
1 module pwm_led(  
2     input wire clk,  
3     input wire xrst,  
4     input wire [9:0] width,  
5     output reg pwm);  
6  
7     reg [9:0] counter;  
8  
9     always @(posedge clk or negedge xrst)  
10    begin  
11        if (!xrst) counter <= 10'd0;  
12        else  
13            begin  
14                counter <= counter + 10'd1;  
15            end  
16        end  
17  
18    always @(posedge clk or negedge xrst)  
19    begin  
20        if (!xrst) pwm <= 1'b0;  
21        else  
22            begin  
23                if (width <= counter) pwm <= 1'b1;  
24                else pwm <= 1'b0;  
25            end  
26        end  
27  
28    endmodule
```

※ カウンターとコンパレータはブロックとしては異なるもの  
なので、異なるalways文で作成しているが、1つのalways文で  
書いても問題ない

パルスを作るカウンターのカウントアップ

コンパレータの動作をするif文

外部からwidthを与えるので、このモジュールを呼び出す  
モジュールを作る

# Verilogコード

```
1 module pwm_top(  
2     input wire clk,  
3     input wire xrst,  
4     output wire led);  
5  
6  
7 module pwm_led l1l(  
8     .clk(clk),  
9     .xrst(xrst),  
10    .width(10'd255),  
11    .pwm(led));  
12  
13 endmodule  
14
```

widthに適当に10ビットの数値を入れる。

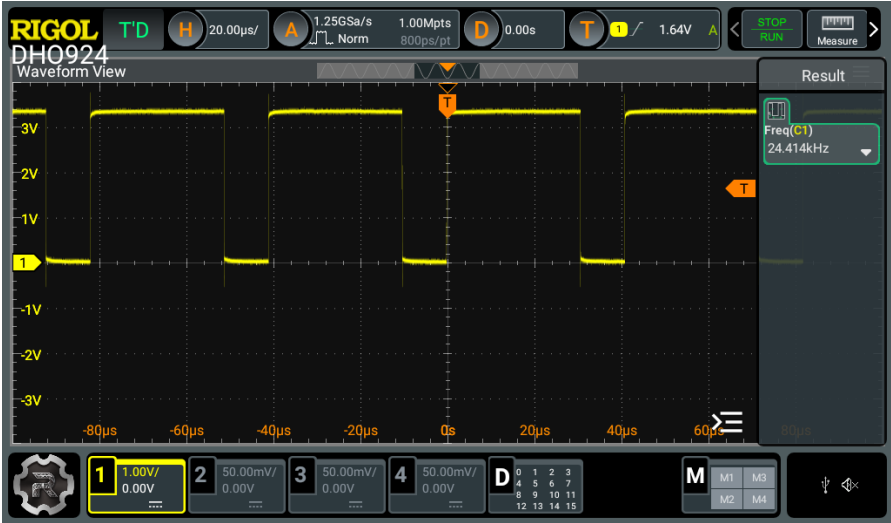


# ピンアサイン

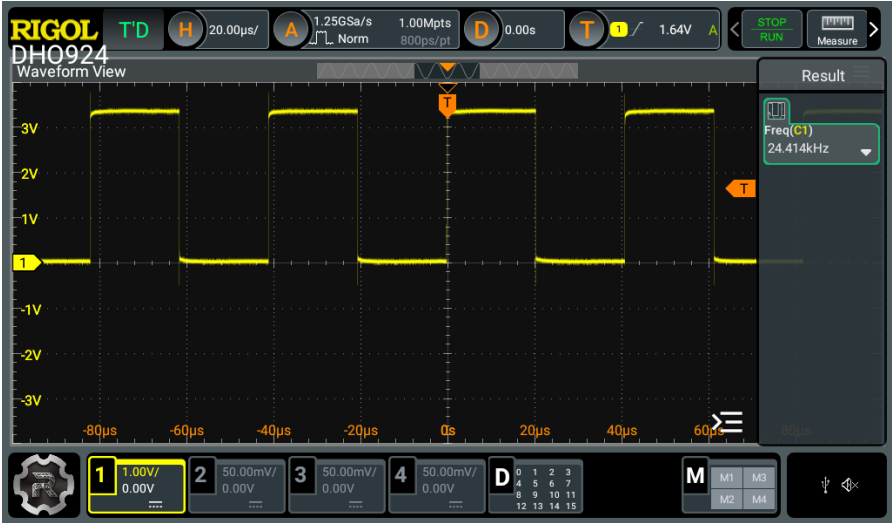
| GPIO : Instance View |             |          |          |          |          |              |                |
|----------------------|-------------|----------|----------|----------|----------|--------------|----------------|
|                      |             |          | all      | all      | all      | all          |                |
| Instance             | Package Pin | Resource | I/O Bank | Alt Conn | Features | Clock Region | Pad            |
| clk                  | C2          | GPIOL_16 | 1B       | GCLK     | None     | L1           | GPIOL_16_CLK2  |
| led                  | D3          | GPIOL_19 | 1B       | GCTRL    | None     | L1           | GPIOL_19_CTRL3 |
| xrst                 | A6          | GPIOR_03 | 2A       | None     | None     | R1           | GPIOR_03       |

動作

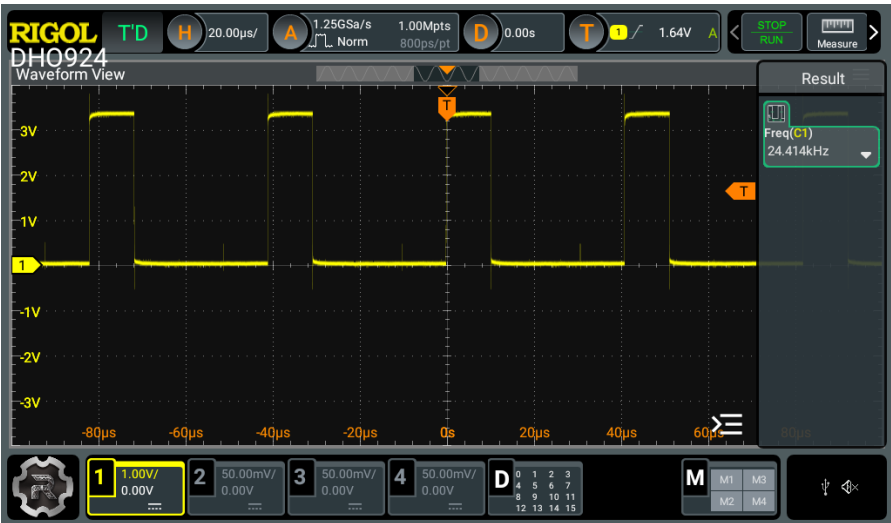
width=10'd255



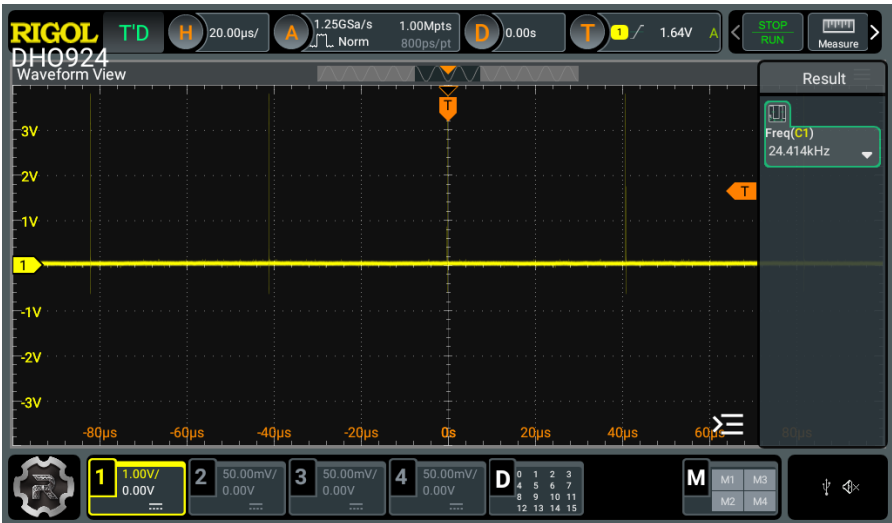
width=10'd511



width=10'd766



width=10'd1023



# Verilogコード

```
1 module pwm_top(  
2   input wire clk,  
3   input wire xrst,  
4   output wire led);  
5  
6  
7   pwm_led l1l(  
8     .clk(clk),  
9     .xrst(xrst),  
10    .width(width),  
11    .pwm(led));  
12  
13   reg [15:0] counter;  
14   reg [9:0] width;  
15  
16  
17   always @(posedge clk or negedge xrst)  
18   begin  
19     if (!xrst)  
20     begin  
21       counter <= 16'd0;  
22     end  
23     else  
24     begin  
25       if (counter < 16'd50000) counter <= counter + 16'd1;  
26       else counter <= 16'd0;  
27     end  
28   end  
29  
30   always @(posedge clk or negedge xrst)  
31   begin  
32     if (!xrst)  
33     width <= 10'd0;  
34     else  
35     if (counter == 16'd25000) width <= width + 10'd4;  
36   end  
37 endmodule  
38
```

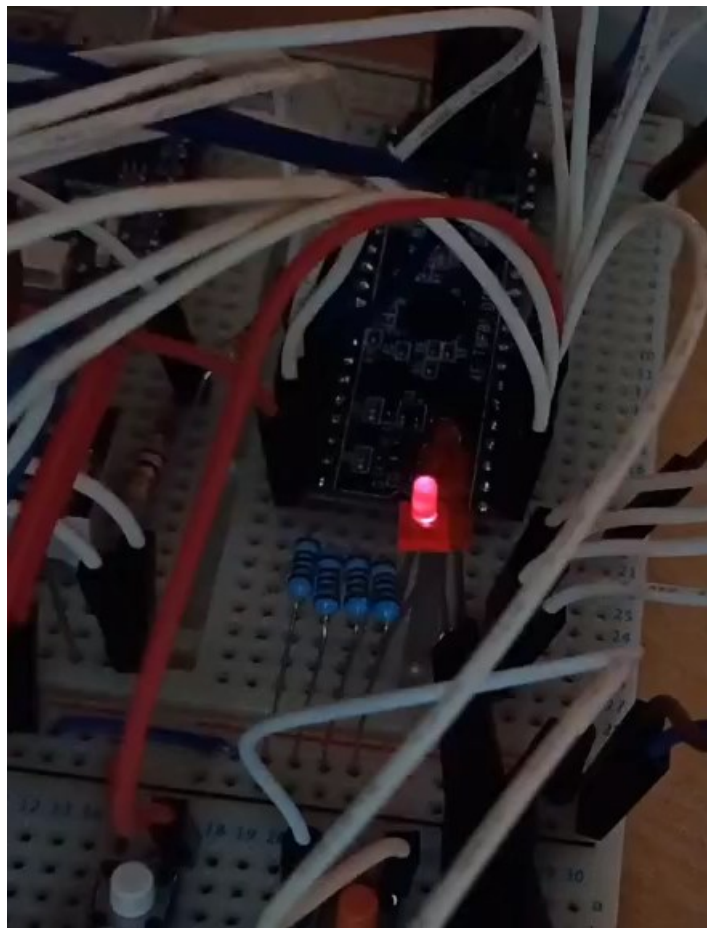
widthの値を一定時間で更新するようにpwm\_topを改良する

widthを更新するタイミングを作るタイマー

widthの更新。タイマーがカウントアップすると、4を加算する

# 動作

12

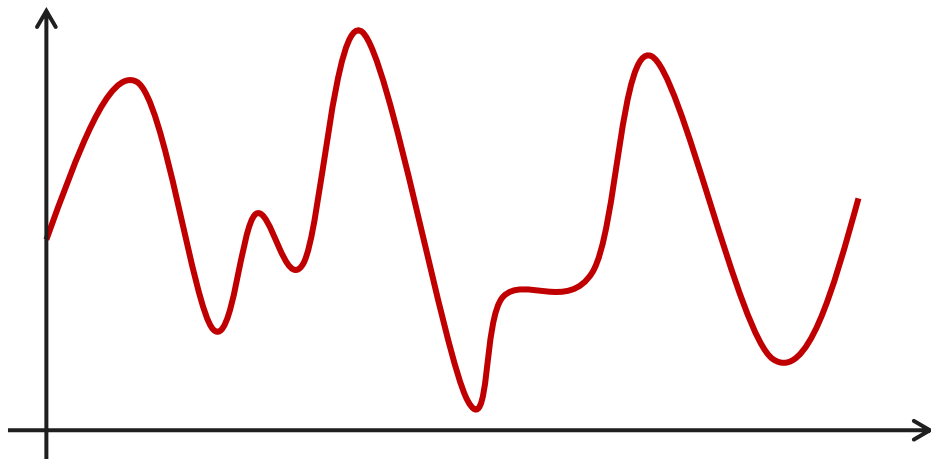


$\Delta \Sigma$ のその前に

# 連続信号と離散信号

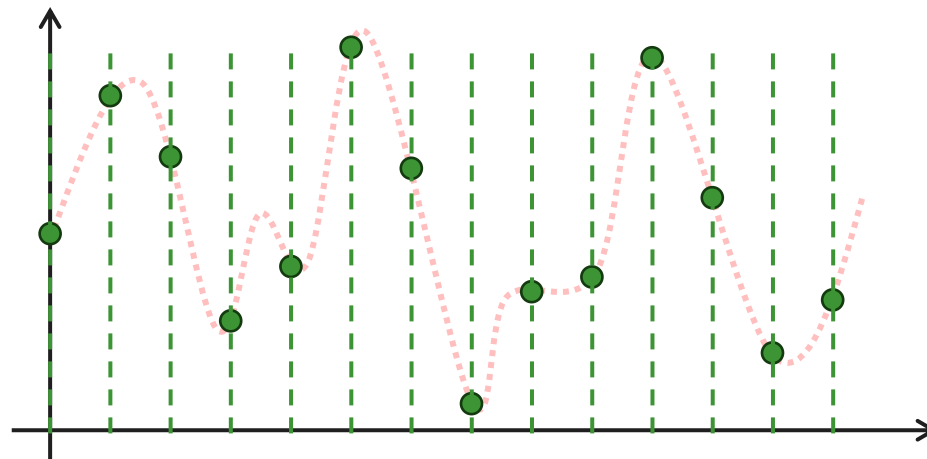
## 連続信号

信号の2点間のどこを取ってもその瞬間の値が決まる信号



## 離散信号

時間方向に分割された値の集合で構成される信号。2点間の値は決めることができない信号



# ナイキスト周波数

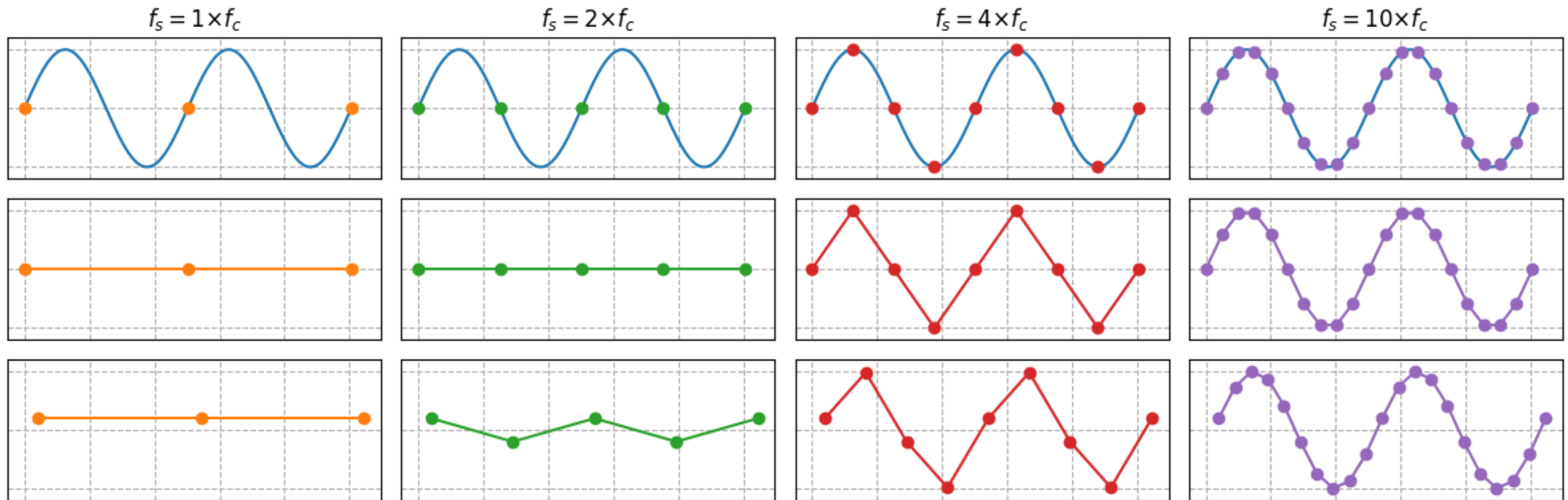
ある周波数の信号を取得するのにどの程度のサンプリング周波数が必要になるのか？

→ 取得する信号周波数の2倍以上の周波数でサンプリングする

$$f_s > 2f_{max}$$

2倍未満では離散化された振幅に周波数情報が残らない

(厳密にはサンプリング周波数を中心として対称の周波数成分として出てくる)

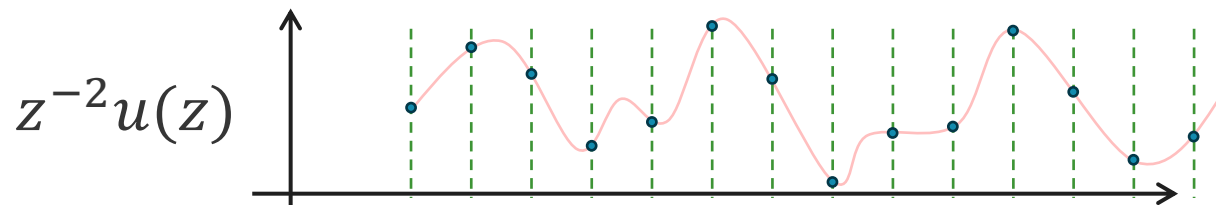
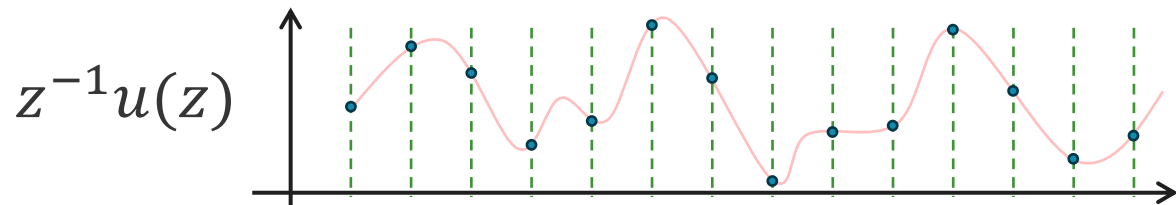
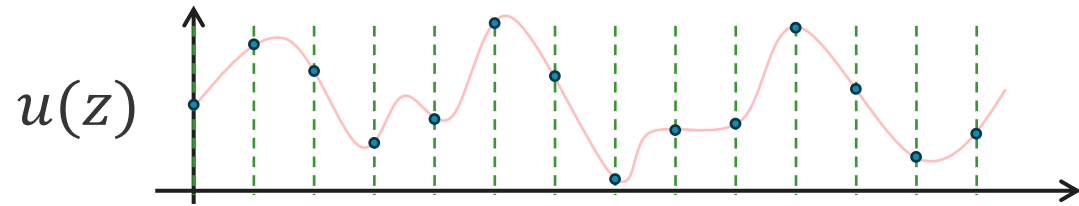


# 遅延要素 $z^{-1}$

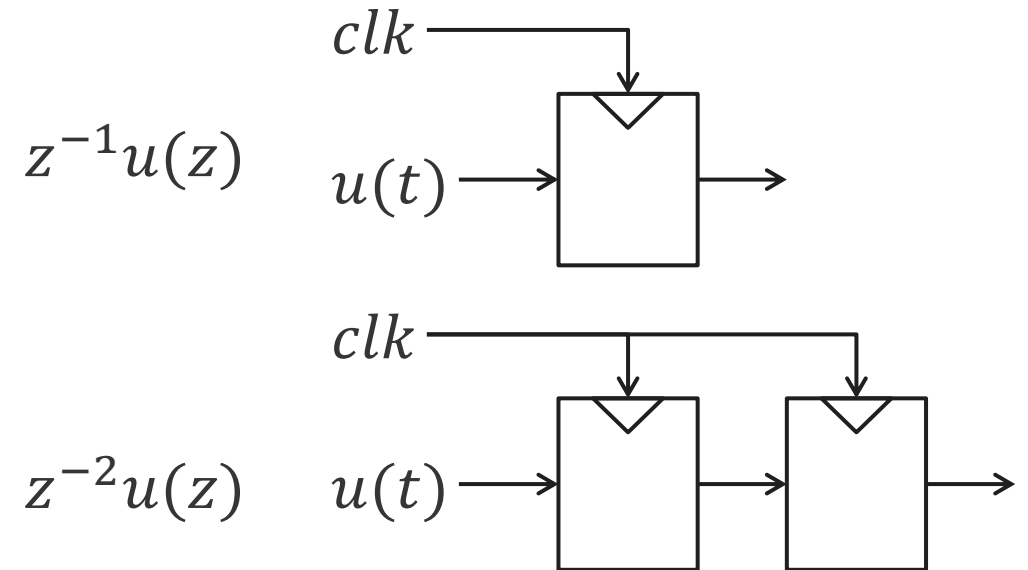
デジタル信号処理では $z$ というものがたくさん出てきます。  
 数学的な詳細なことは信号処理の本を読んでもらうとして...

$z^{-1}$ は信号を一単位時間遅延させることを意味します。

右肩の数字は遅延させる時間を意味します。 $z^{-2}$ は二単位時間遅延します。



$z$ はデジタルではD-FFに相当します。





$\Delta \Sigma$  変調

# まず初めに $\Delta$ があった

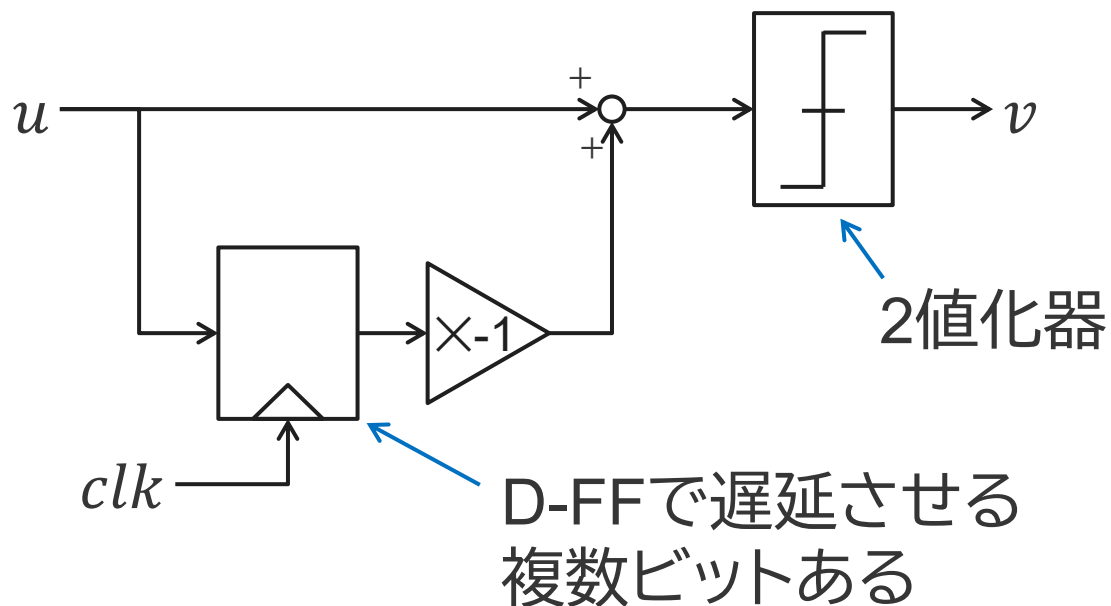
十分に早くサンプリングしたデータは隣り合う値の変化は小さい  
→ 差分( $\Delta$ )だけを扱うとデータを圧縮することができる

差分( $\Delta$ )が1ビットしか変化しないようであれば、2値化しても問題なくなる

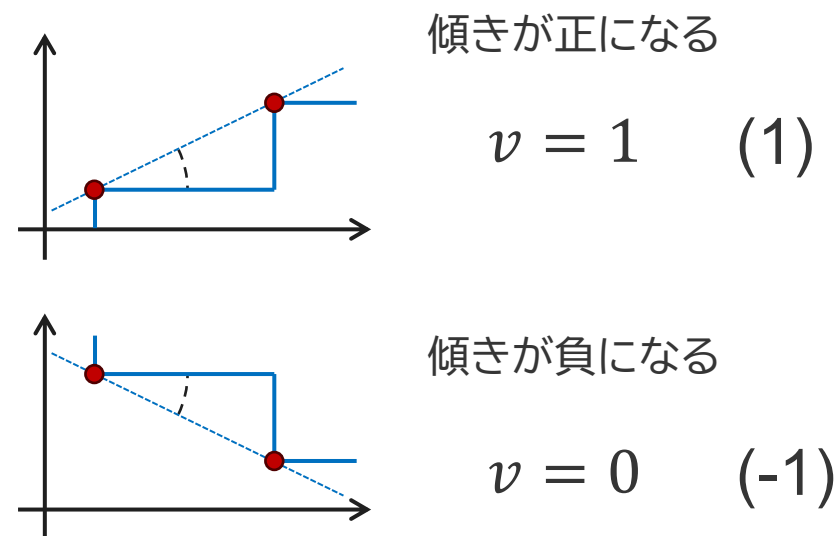
→ 差分の傾きの情報のみを $v$ として圧縮する

→ 差分が1ビットになるように高速にサンプリングする(オーバーサンプリング)

## $\Delta$ を使った送信機

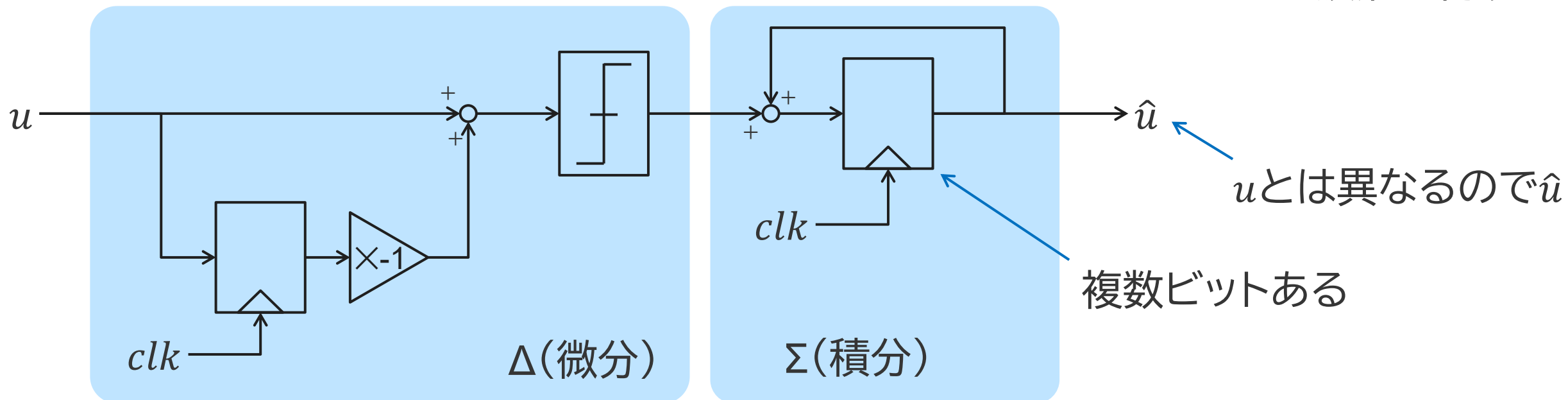
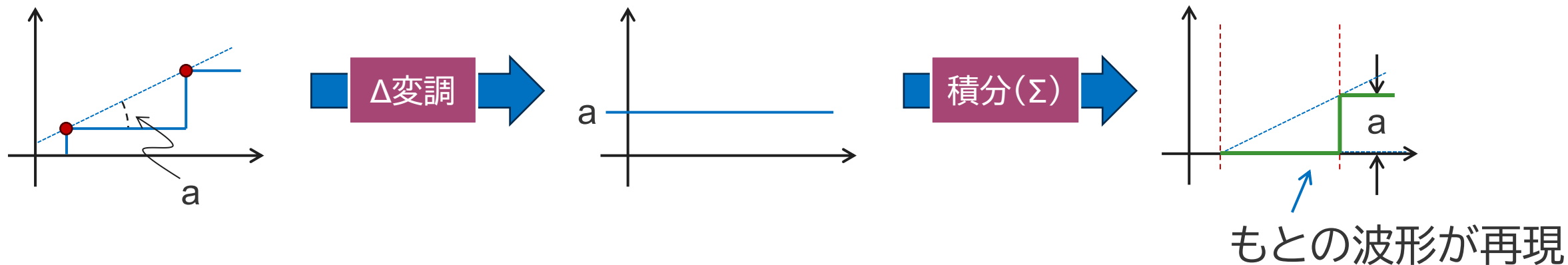


## $v$ の取りうる値



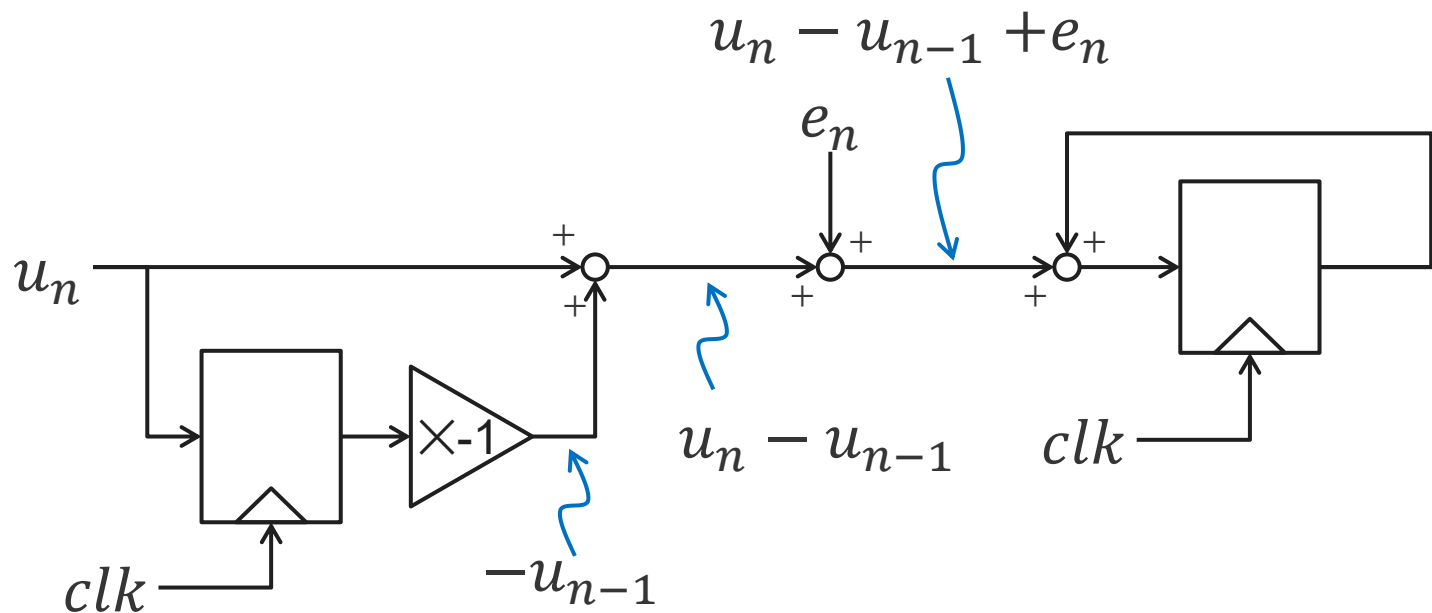
# $\Delta$ は微分なのでもとに戻す積分( $\Sigma$ )がついた

$\Delta$ は差分を取る処理→もとに戻すには差分を足し続ける必要がある



# もう少し考えてみる

2値化器は誤差 $e$ を与えるものとして考えてみる



$$\sum u_{n-1} - u_{n-2} + e_{n-1}$$

$$= u_{n-1} - u_m + \sum e_{n-1}$$

$$\begin{array}{l} u_{n-1} - u_{n-2} + e_{n-1} \\ u_{n-2} - u_{n-3} + e_{n-2} \\ u_{n-3} - u_{n-4} + e_{n-3} \end{array}$$

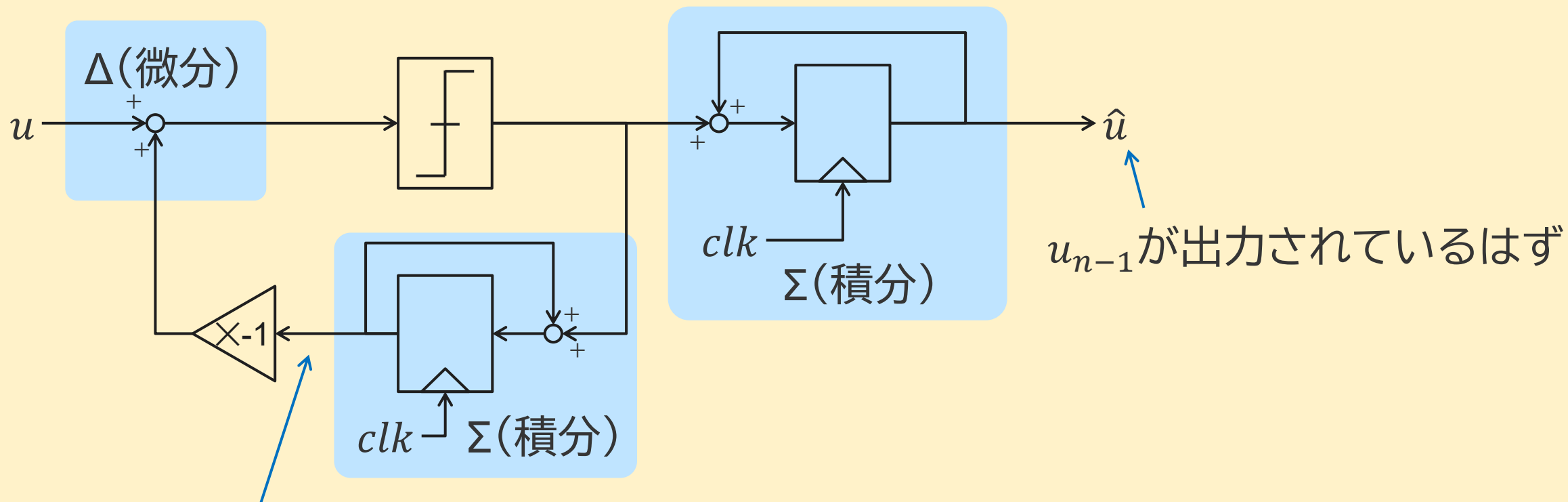
マイナスの項が打ち消されるので  
確かに $u_{n-1}$ に戻っている

しかし、2値化器の誤差 $\sum e_{n-1}$ が残ってしまう

# 2値化器の誤差を打ち消して伝送したくなった

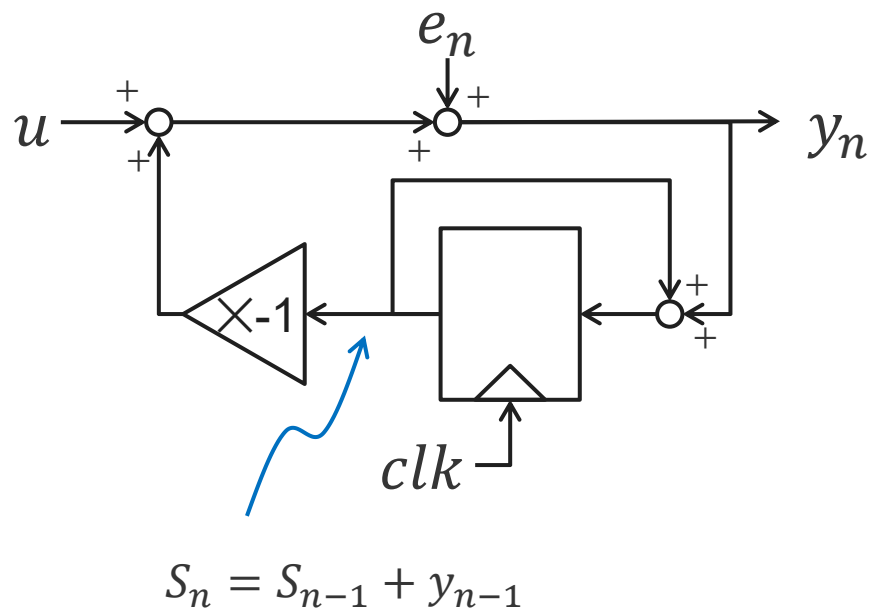
積分( $\Sigma$ )した結果で差分( $\Delta$ )を取れば2値化器で生じる誤差が打ち消せるはず

## $\Delta$ 変調器



積分結果は $u_{n-1}$ と誤差なのでこれでも問題ない

# Δ変調器をもう少し考える1



$y_n$ がどのようなになるか考える。

$$y_n = u_n - s_n + e_n = u_n - (s_{n-1} + y_{n-1}) + e_n$$

ここで $y_{n+1}$ を考える。

$$y_{n+1} = u_{n+1} - s_{n+1} + e_{n+1} = u_{n+1} - (s_n + y_n) + e_{n+1}$$

$y_{n+1}$ と $y_n$ の差分を取る。

$$\begin{aligned} y_{n+1} - y_n &= u_{n+1} - u_n - s_n + s_{n-1} - y_n + y_{n-1} + e_{n+1} - e_n \end{aligned}$$

$s_n = s_{n-1} + y_{n-1}$ を代入。

$$\begin{aligned} y_{n+1} - y_n &= u_{n+1} - u_n - (s_{n-1} + y_{n-1}) + s_{n-1} - y_n + y_{n-1} + e_{n+1} - e_n \\ &= u_{n+1} - u_n - y_n + e_{n+1} - e_n \end{aligned}$$

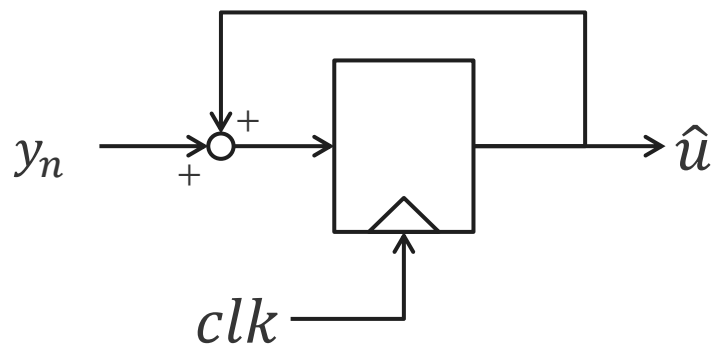
$y_{n+1} = u_{n+1} - u_n + e_{n+1} - e_n$ が得られる。 $n+1$ は $n$ にできるので、

$y_n = u_n - u_{n-1} + e_n - e_{n-1}$ となる。

ここからΔ変調器は信号だけでなくエラーも微分することがわかる。

# Δ変調器をもう少し考える2

受信機の積分器について見る



$\hat{u}$ は積分器の出力なので、 $y_n$ を加算することを考えると

$$\begin{aligned}
 y_n &= u_n - \cancel{u_{n-1}} + e_n - \cancel{e_{n-1}} \\
 y_{n-1} &= \cancel{u_{n-1}} - \cancel{u_{n-2}} + \cancel{e_{n-1}} - \cancel{e_{n-2}} \\
 y_{n-2} &= \cancel{u_{n-2}} - \cancel{u_{n-3}} + \cancel{e_{n-2}} - \cancel{e_{n-3}} \\
 y_{n-3} &= \cancel{u_{n-3}} - \cancel{u_{n-4}} + \cancel{e_{n-3}} - \cancel{e_{n-4}} \\
 y_{n-4} &= \cancel{u_{n-4}} - \cancel{u_{n-5}} + \cancel{e_{n-4}} - \cancel{e_{n-5}}
 \end{aligned}$$

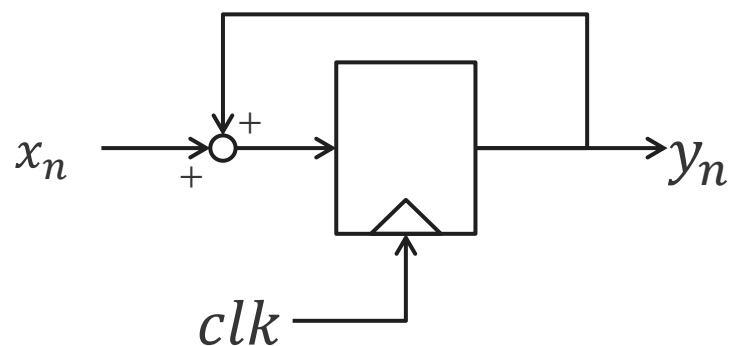
となっていくので、結果として $\hat{u} = u_n + e_n$ が出力される。

このようにΔ変調器では $e_n$ によるDCオフセットを生じることが避けられず、DC入力に対して脆弱である。他にフィードバックパスに積分器があることでその非理想要因が正確性を劣化させる。

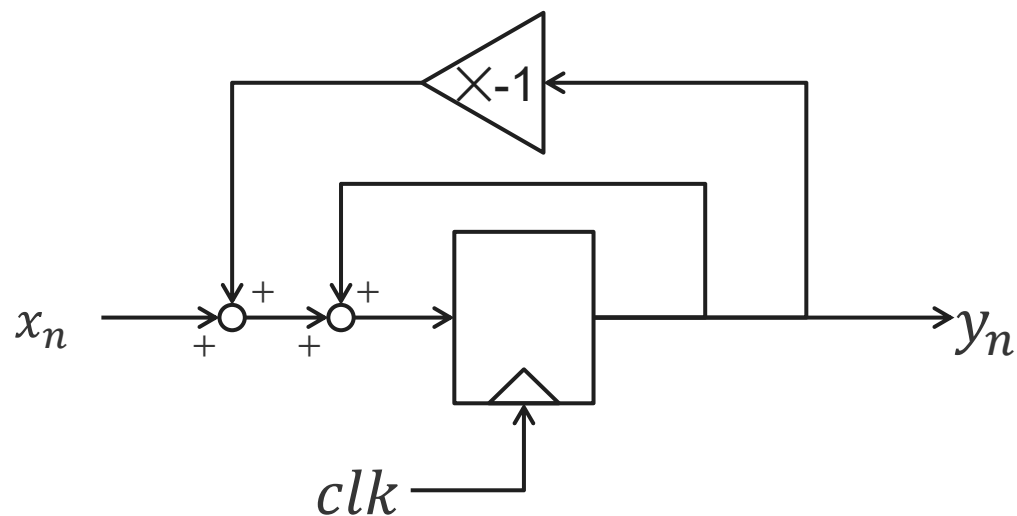
→これらを改善したのがΔΣ変調器。

# $\Delta$ 変調器の積分器の位置を更に変える

変調器の出力が微分したのではなく、積分まで済ませたものにできないか？



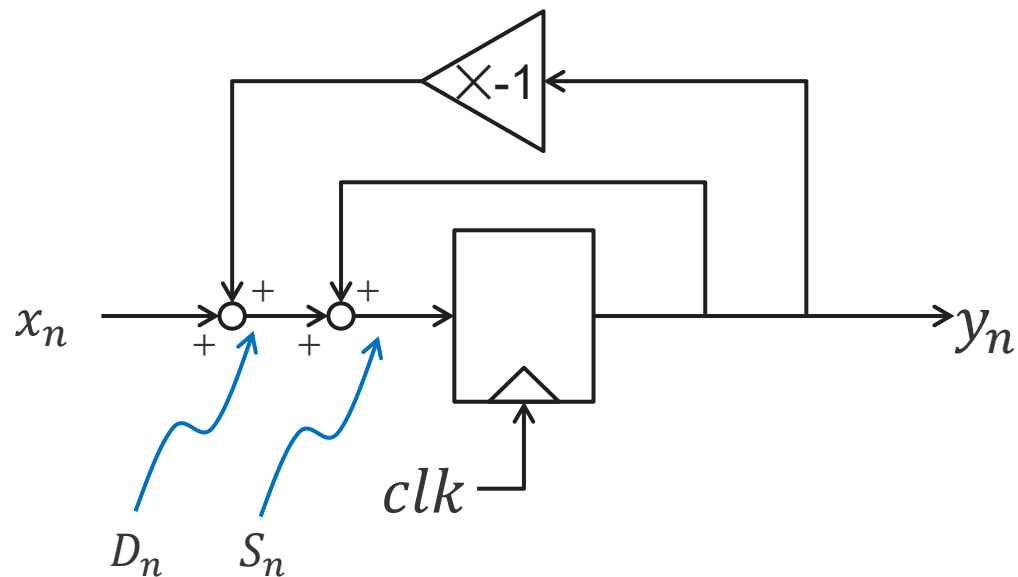
左の積分器のD-FFを遅延素子として、  
微分器を構成してみる



積分器( $\Sigma$ )を取り囲むように微分器( $\Delta$ )を挿入する。  
→実はこれに量子化器(2値化器)をつけることで $\Delta\Sigma$ 変調器になる。



# 量子化器の無い $\Delta\Sigma$ を考える



$\Delta$ のための加算器を $D_n$ 、シグマのための加算機は $S_n$ とする。そうすると、

$$D_n = x_n - y_n$$

$$S_n = D_n + y_n$$

となる。また $y_n$ は $S_n$ を1つ遅延させたものなので、

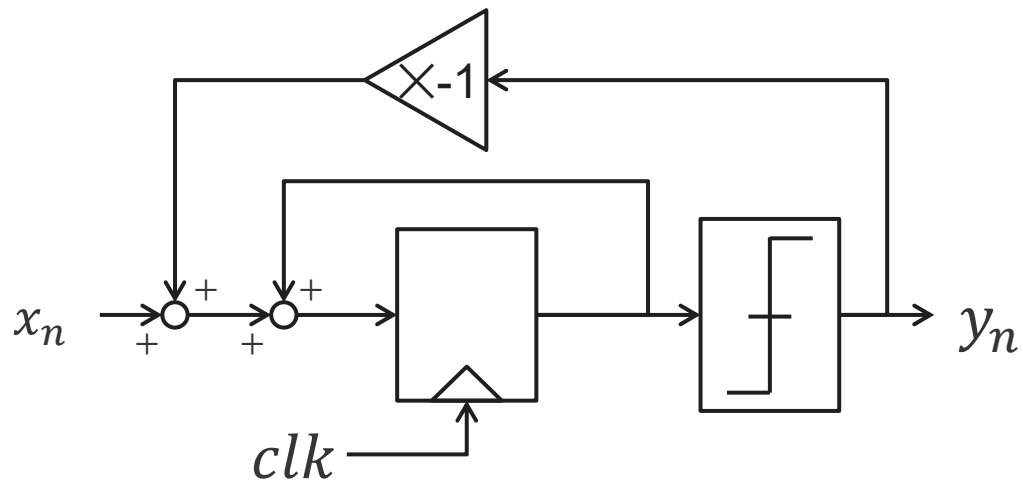
$$y_n = S_{n-1}$$

したがって、

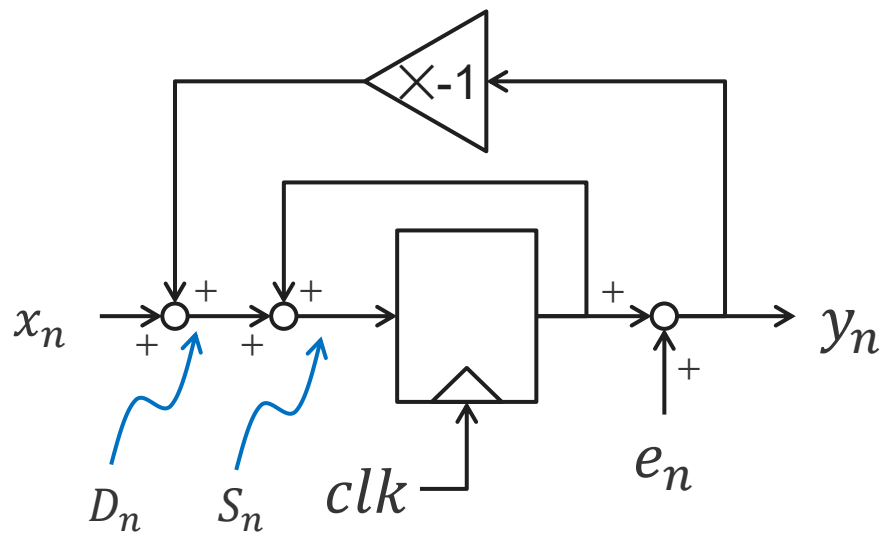
$$y_n = x_{n-1}$$

となり、微分したものを積分するので、当たり前だが1つ遅延したものが出力される。

# $\Delta \Sigma$ 変調器

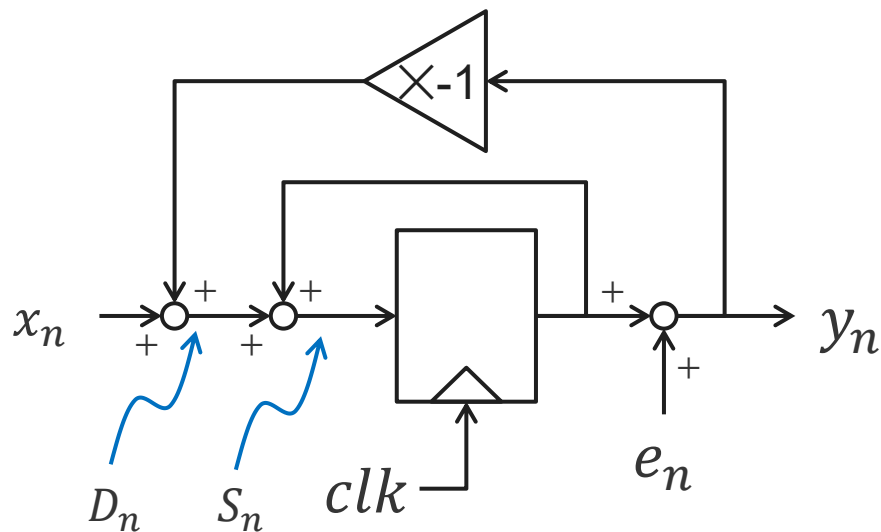


$\Delta$ 変調器と同様に2値化した結果の差分を取る構成に変更する。



解析のために2値化器は量子化誤差をエラーとして加算するブロックとして置き換えてその動作を確認する。

# ΔΣ変調器の解析



ブロックから数式を起こす。まず、

$$D_n = x_n - y_n$$

となる。次に $S_n$ は1つ遅延した値 $S_{n-1}$ を用いて

$$S_n = D_n + S_{n-1}$$

となる。 $y_n$ は、

$$y_n = S_{n-1} + e_n$$

となる。 $y_n$ を $D_n$ に代入すると、

$$D_n = x_n - (S_{n-1} + e_n)$$

である。これを $S_n$ に代入すると、

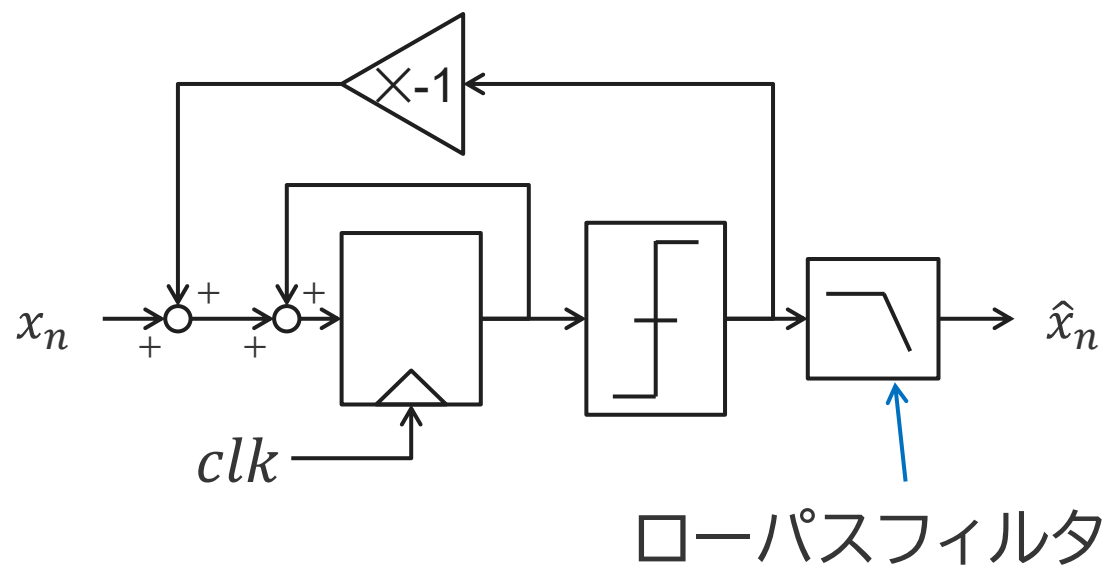
$$S_n = x_n - S_{n-1} - e_n + S_{n-1} = x_n - e_n$$

となる。これを $y_n$ に代入すると、

$$y_n = x_{n-1} + e_n - e_{n-1}$$

となる。このようにΔΣ変調器は、入力を1つ遅延したものと、量子化誤差の微分が出力される。

# $\Delta\Sigma$ 変調器の出力から元の信号を得る

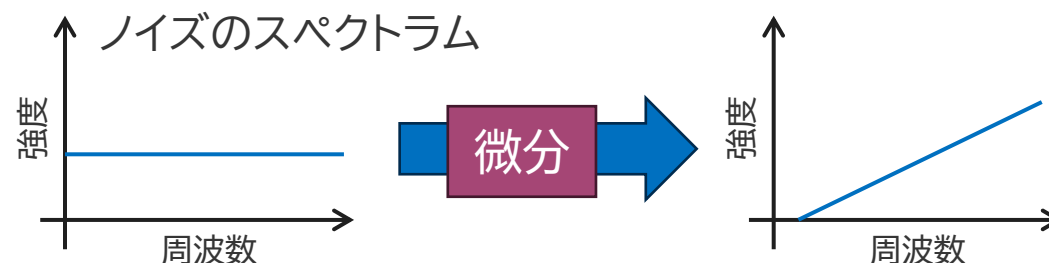


$\Delta\Sigma$ 変調器の出力は

- 元の信号を遅延したもの
  - 量子化誤差の微分
- が出力される。

→つまり、出力は何もなくても元の信号を含む信号が出ている。

量子化誤差の微分がLPFで削れる？  
量子化誤差は周波数軸に一様に分布する。



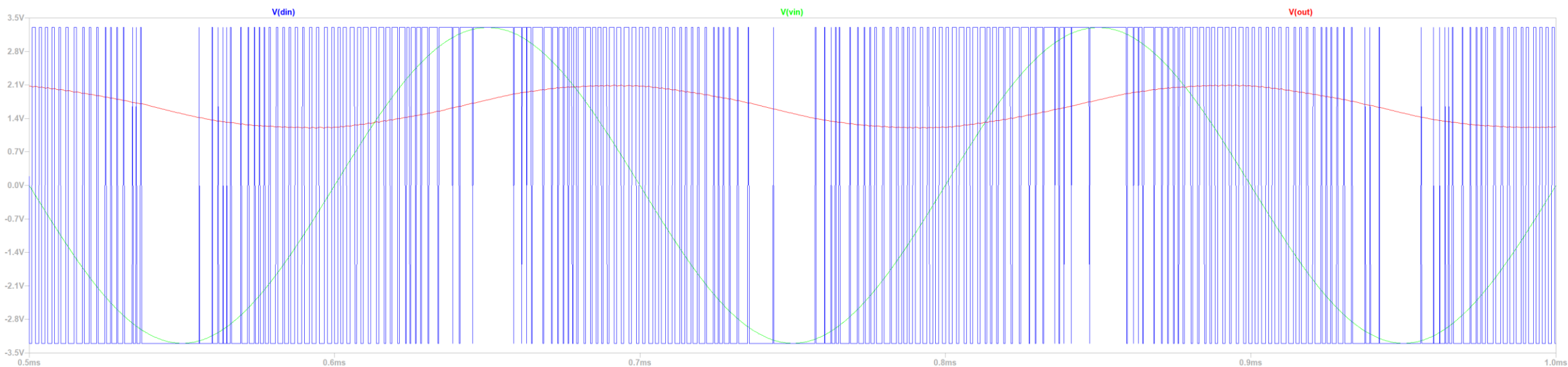
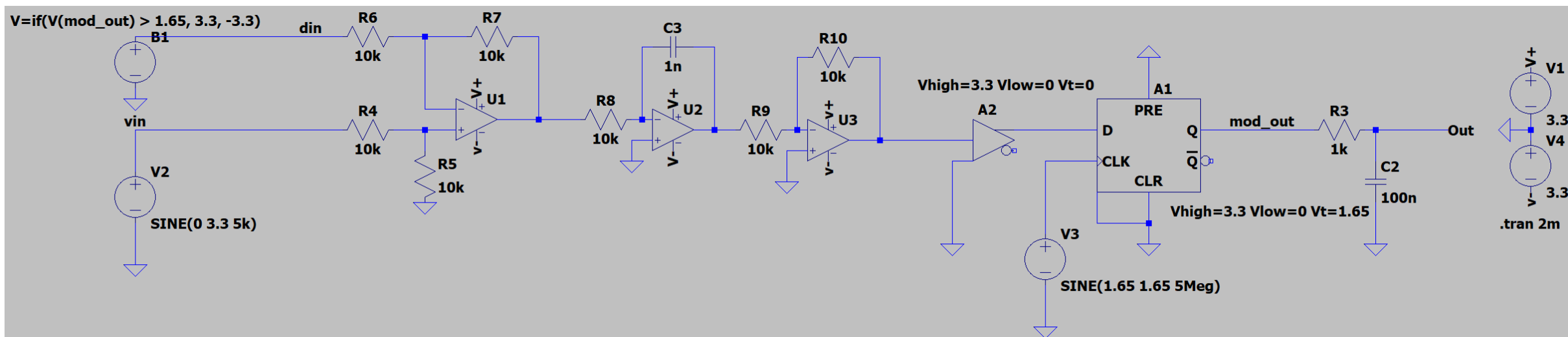
ある信号  $s(f) = \sin(2\pi f t)$  の微分は

$$\frac{ds}{dt} = 2\pi f \cos(2\pi f t)$$

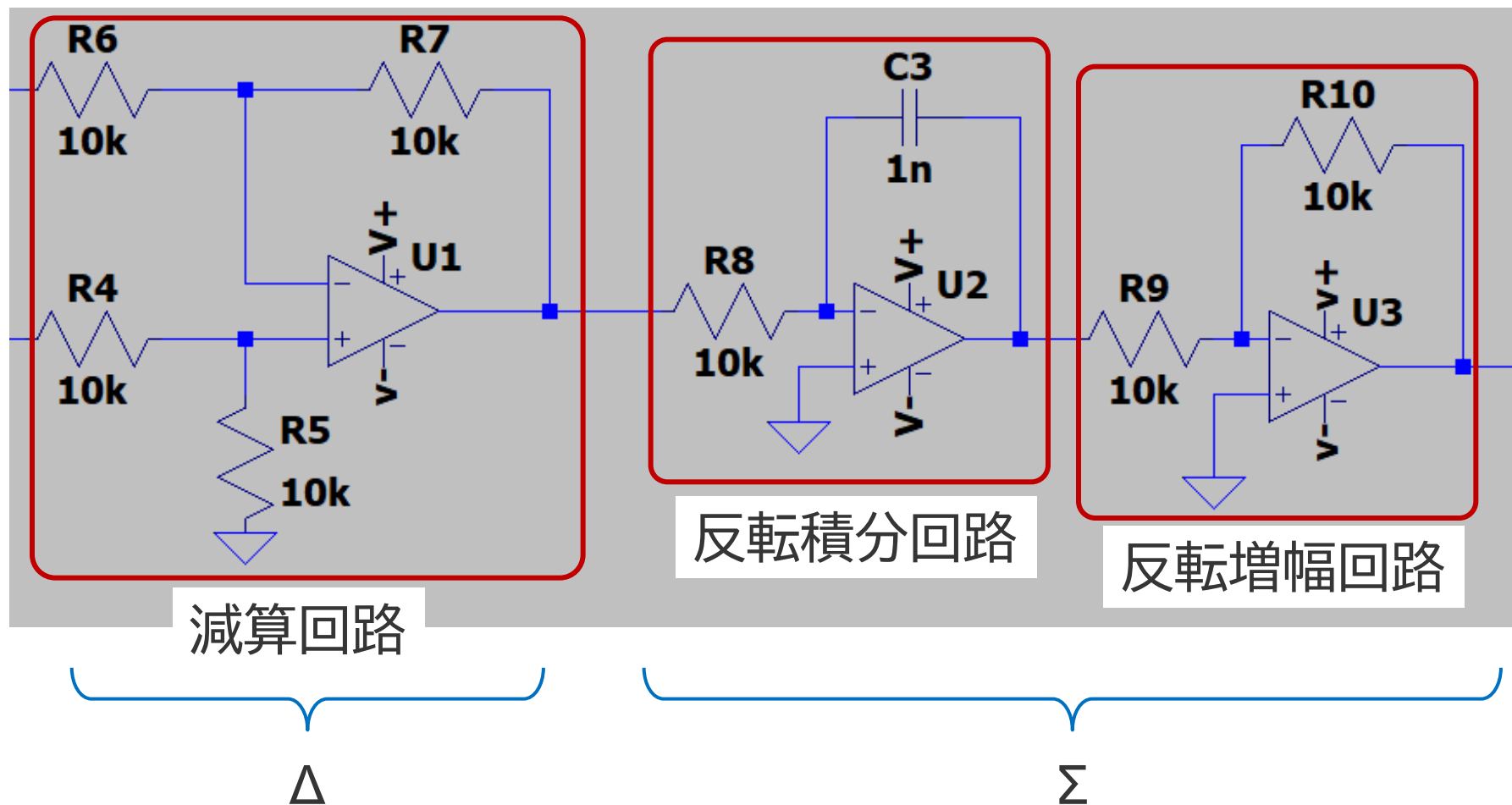
となり、周波数が大きくなると強調される。  
一方で低域は小さくなるので、LPFで高周波のノイズ成分を落とせば低ノイズに信号を再生することができる。  
→これをノイズシェーピングという。

# $\Delta \Sigma$ 変調のシミュレーション

Itspiceを用いてシミュレーションしてみる

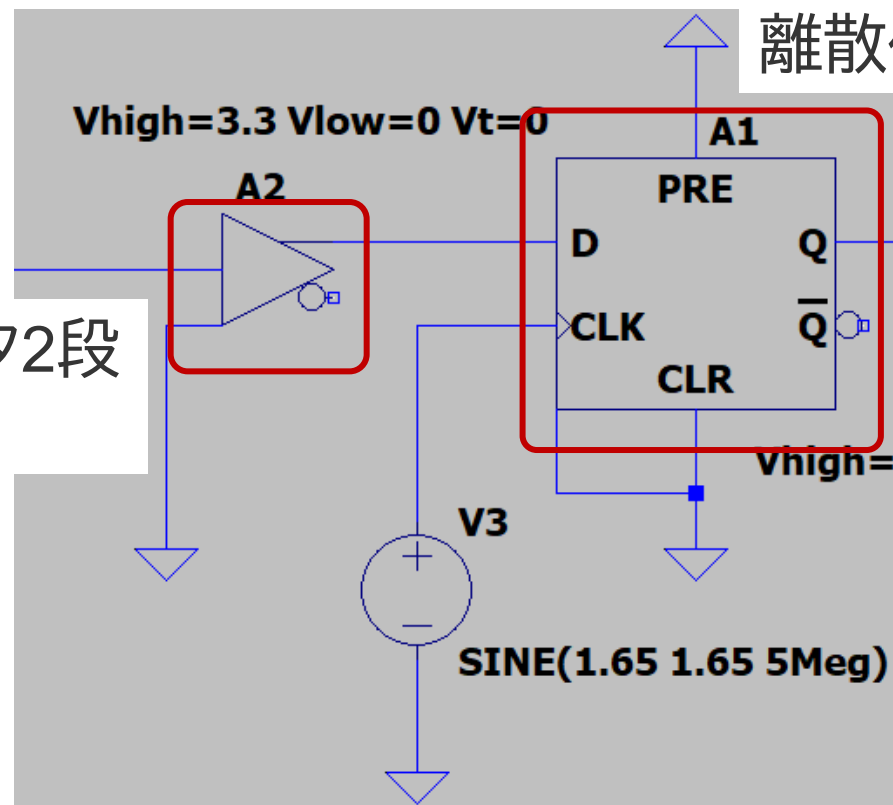


# 回路をよく見てみる1



# 回路をよく見てみる2

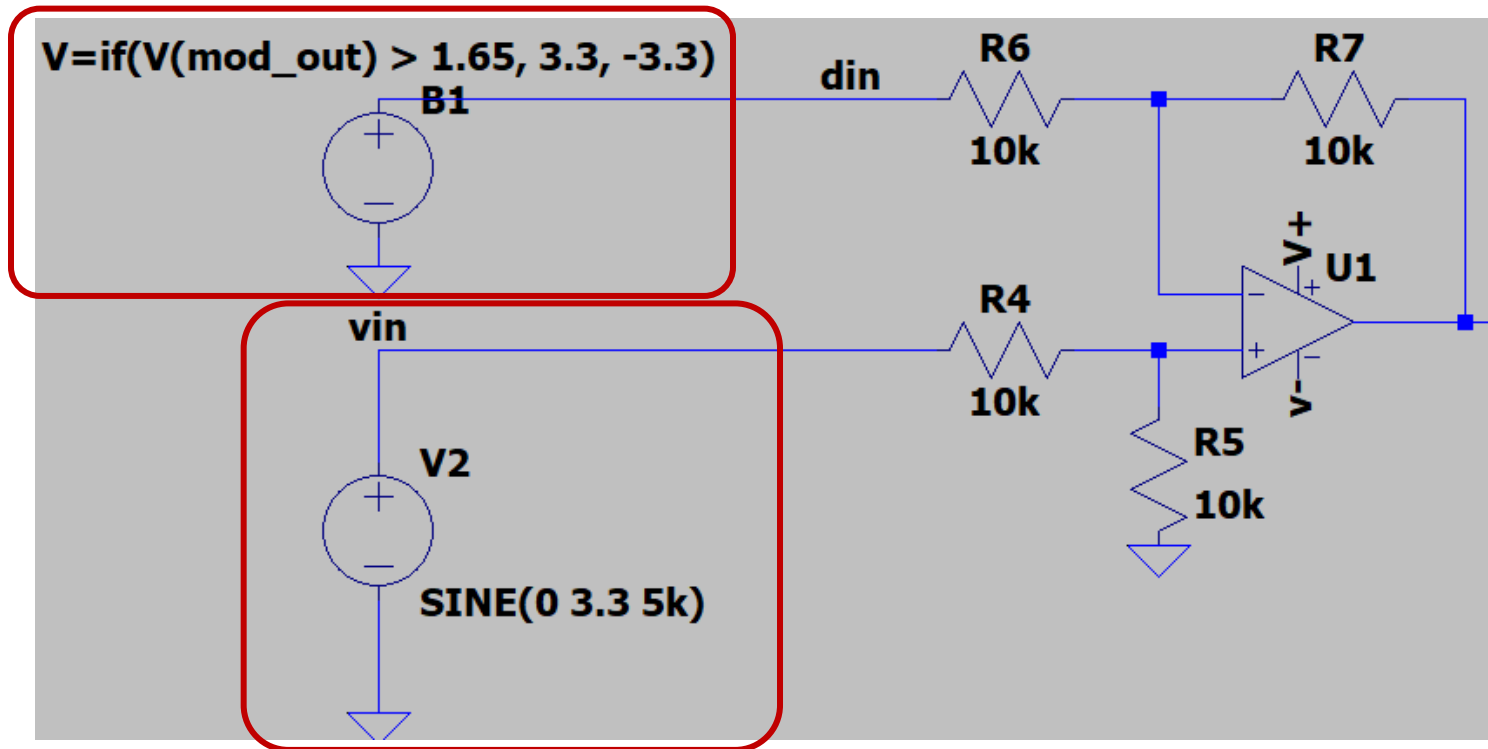
バッファ回路=インバータ2段  
→2値化器



D-FF→積分器が連続なので、  
離散化するのに使う

# 回路をよく見てみる3

D-FFの出力を0と1から-3.3と3.3に変換するビヘイビアボルテージソース

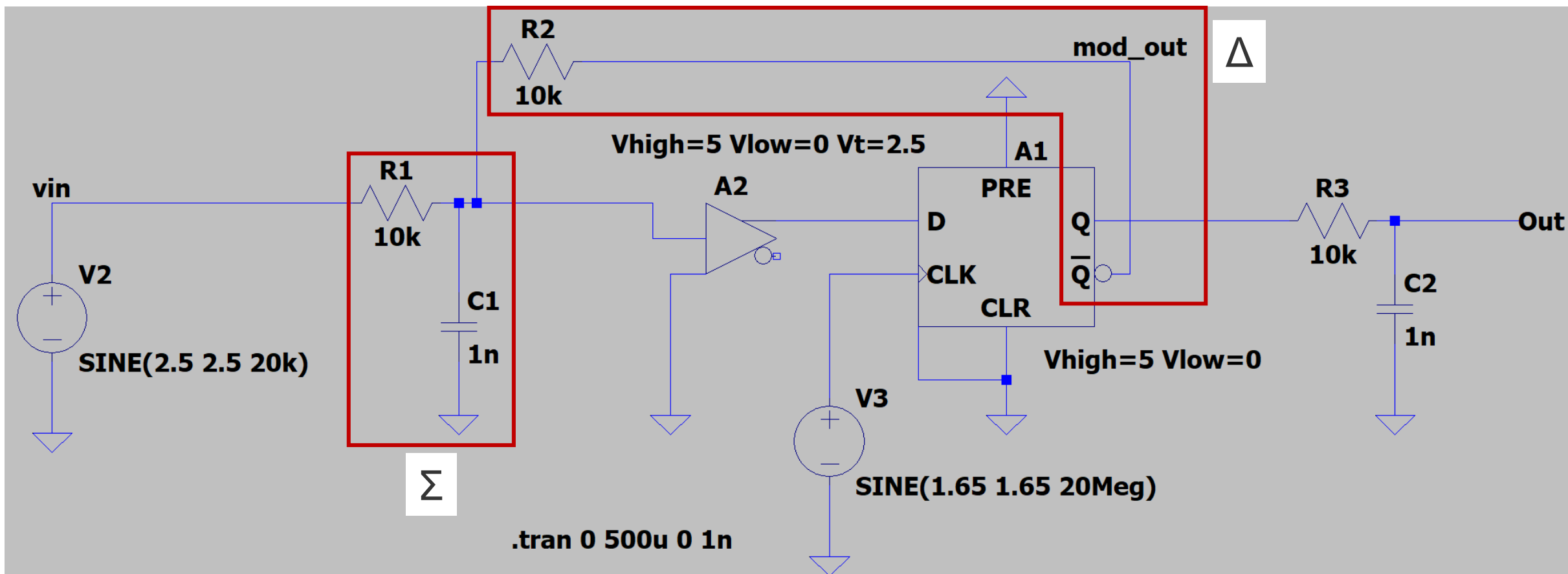


入力信号

オペアンプを3個も使うのでもっと簡単な回路で実現したい！

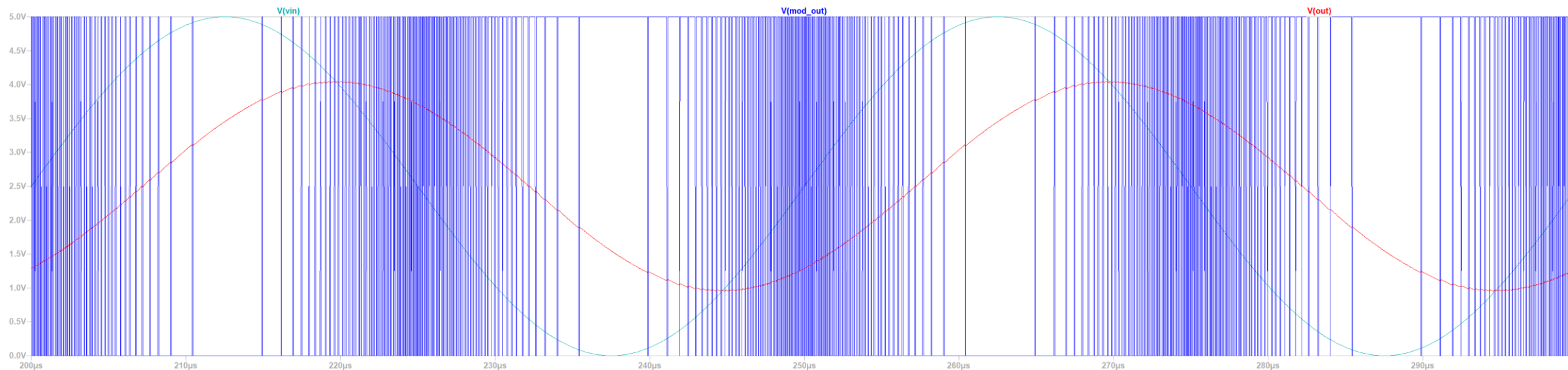


# パッシブ素子だけで $\Delta\Sigma$



- $\Sigma$ はRCローパスフィルタ
- $\Delta$ はD-FFの反転出力と抵抗

# シミュレーション結果

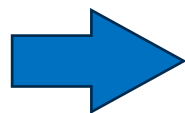


パッシブ素子だけでも $\Delta\Sigma$ 変調ができている

- $\Sigma$ はRCローパスフィルタ
- $\Delta$ はD-FFの反転出力と抵抗

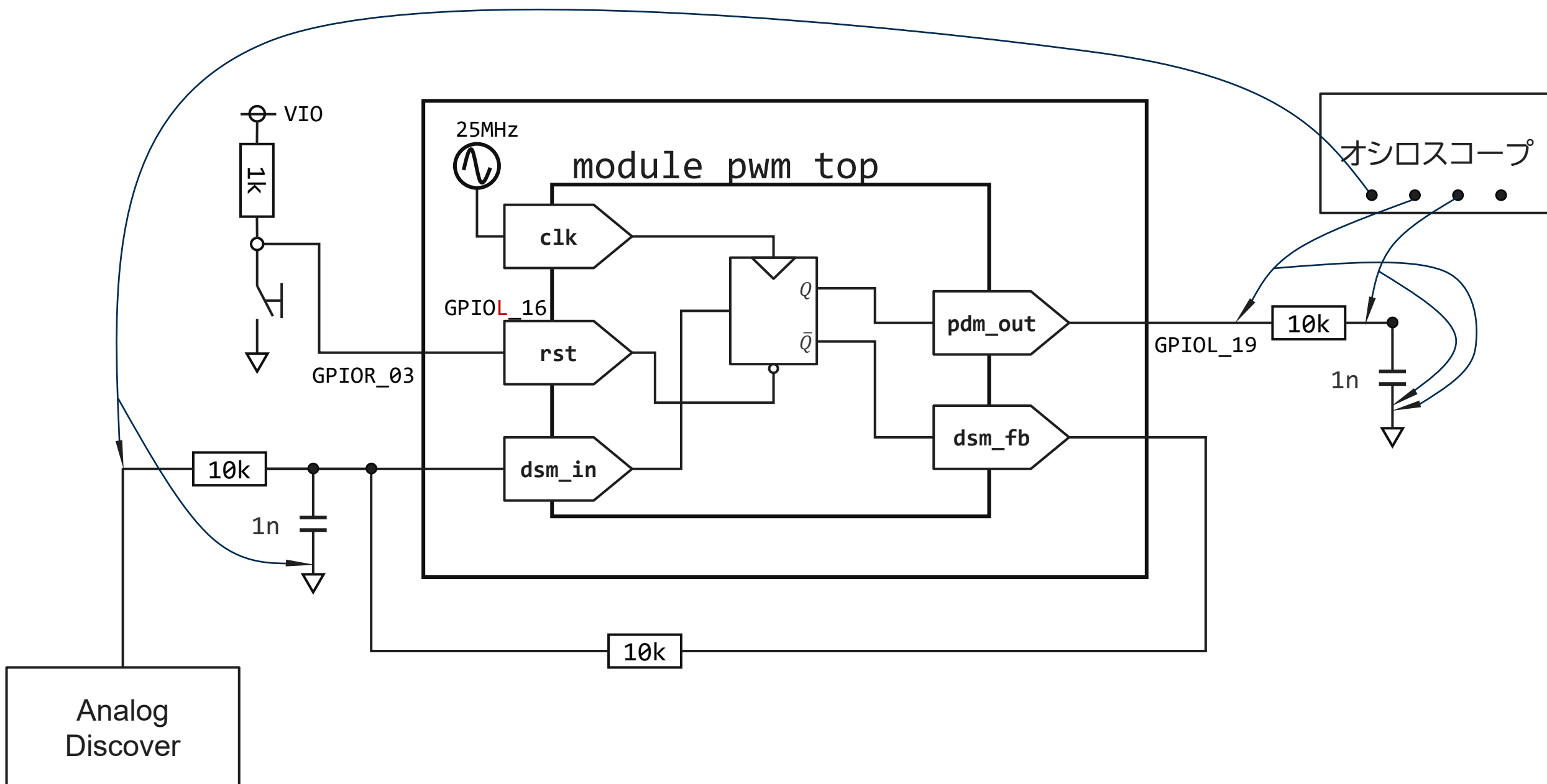
+

- コンパレータはIO端子
- D-FFはレジスタ



**FPGAでも実現できる**

# FPGAで $\Delta\Sigma$ 変調



# Verilogコード

```
1 module dsm_adc(  
2     input wire clk,  
3     input wire xrst,  
4     input wire dsm_in,  
5     output reg dsm_fb,  
6     output reg pdm_out);  
7  
8     always @(posedge clk or negedge xrst)  
9     begin  
10         if (!xrst)  
11         begin  
12             dsm_fb <= 1'b0;  
13             pdm_out <= 1'b0;  
14         end  
15         else  
16         begin  
17             dsm_fb <= ~dsm_in;  
18             pdm_out <= dsm_in;  
19         end  
20     end  
21  
22 endmodule
```

フィードバックは反転して出力

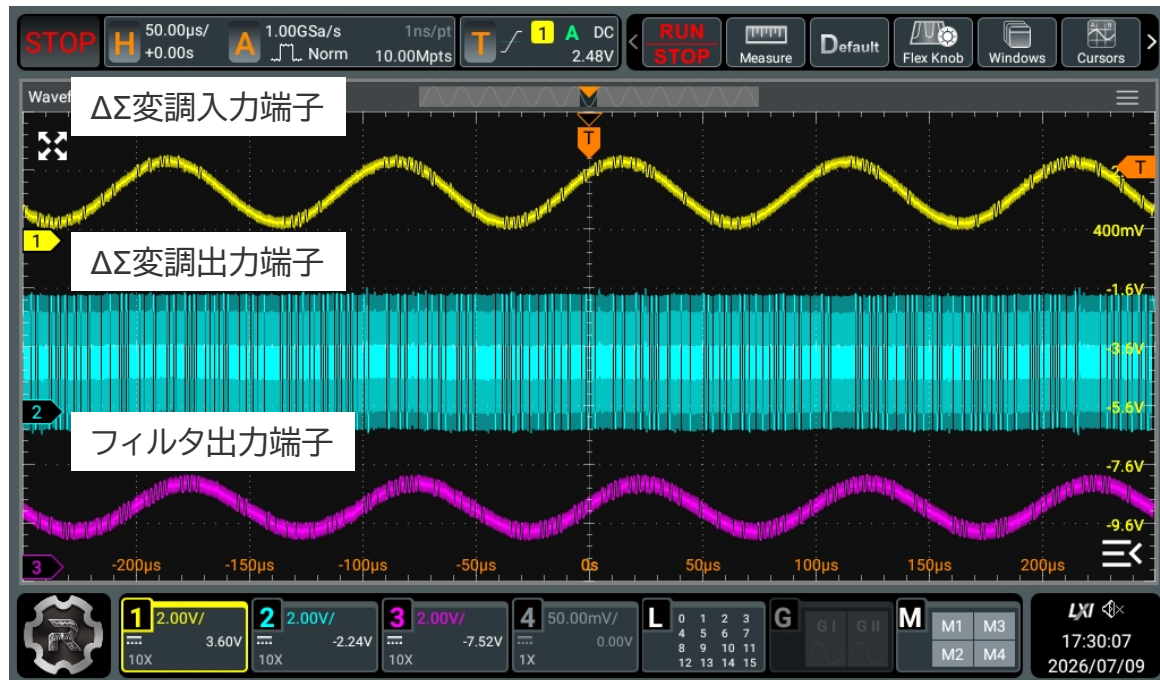
$\Delta\Sigma$ 出力はそのまま出力

# ピンアサイン

GPIO : Instance View

| Instance | Package Pin | Resource | I/O Bank | Alt Conn | Features | Clock Region | Pad            |
|----------|-------------|----------|----------|----------|----------|--------------|----------------|
| clk      | C2          | GPIOL_16 | 1B       | GCLK     | None     | L1           | GPIOL_16_CLK2  |
| dsm_fb   | F1          | GPIOL_13 | 1A       | GCTRL    | None     | L0           | GPIOL_13_CTRL1 |
| dsm_in   | E2          | GPIOL_14 | 1A       | GCLK     | None     | L0           | GPIOL_14_CLK0  |
| pdm_out  | G1          | GPIOL_12 | 1A       | GCTRL    | None     | L0           | GPIOL_12_CTRL0 |
| xrst     | A6          | GPIOR_03 | 2A       | None     | None     | R1           | GPIOR_03       |

# FPGAの $\Delta\Sigma$ 変調と復調結果



- 入力のサイン波が $\Delta\Sigma$ 変調によりパルスの密度に変換されている事がわかる。
- $\Delta\Sigma$ 変調出力をフィルタリングすることで元の波形が再現されていることがわかる。

➡ 2値の信号でもアナログ波形を表現できた！

※波形がノージーなのは $\Delta\Sigma$ のクロックが25MHzで10cm程度のワイヤで配線しているためです。  
ノイズが巻き散らかされてそれが他のワイヤにノイズを誘導するため、ノージーになっています。

# まとめ

- $\Delta$ 変調を説明しその欠点を克服した $\Delta\Sigma$ 変調を説明しました。
- $\Delta\Sigma$ 変調をFPGAといくつかのパッシブ部品で実装しました。
- 次回は $\Delta\Sigma$ 変調で2値になった波形から多値のADCの結果を得る方法から解説します。